

Analysis of the Racetrack Problem as a Heuristic Search Benchmark

by

Valareza Arezehgar

BACHELOR OF SCIENCE

in

The Faculty of Science

(Computer Science)

TORONTO METROPOLITAN UNIVERSITY

(Toronto)

April 2025

Abstract

The racetrack problem is a state-space search problem benchmark that is often used in heuristic search studies. Other papers that use this domain have defined the problem informally, making the benchmark difficult to standardize. This thesis aims to formally define and empirically evaluate the racetrack problem to provide a uniform benchmark for future research. In particular, we formalize the implementation details for the domain's states, actions, transitions, and goal tests. We then delineate the two variants of the problem: New Hampshire (where collisions are prohibited) and Airbag. Next, we provide rigorous definitions for the heuristics used in this domain, as well as two new heuristics that generalize well to the problem. For all heuristics, we provide proofs of admissibility and consistency. Finally, we assess the problem variants and heuristics across multiple maps using both A^* and Weighted A^* . We examine the results for both algorithms to understand the behaviors of the racetrack heuristics and variants.

Acknowledgments

I would like to extend my sincere gratitude to my supervisor, Dr. Richard Valenzano, for his unwavering support throughout this project. His guidance, insights, and assistance were crucial to this thesis and to my growth as a scientist.

Table of Contents

Abstract	ii
Acknowledgments	iii
Table of Contents	iv
List of Figures	vi
Chapter 1: Introduction	1
Chapter 2: Background	3
2.1 State-Space Search	3
2.2 Best-First Search	4
2.3 Heuristics	6
2.4 The A* Algorithm	7
2.5 Weighted A* (WA*)	8
Chapter 3: Formulating The Racetrack Problem	9
3.1 State	9
3.2 Actions	10
3.3 Transition Function	10
3.3.1 Collision Detection	11
3.3.2 The New Hampshire Variant	13
3.3.3 The Airbag Variant	14
3.4 Goal Test	14
Chapter 4: Heuristics	16
4.1 Existing Heuristics	16
4.1.1 Euclidean Heuristic	17
4.1.2 Manhattan Heuristic	18
4.2 Velocity Heuristics	20

TABLE OF CONTENTS

4.2.1	Euclidean Velocity Heuristic	20
4.2.2	Manhattan Velocity Heuristic	22
4.3	New Heuristics Based on Chebyshev Distance	24
4.3.1	Chebyshev Heuristic	24
4.3.2	Chebyshev Velocity Heuristic	25
Chapter 5: Experiments		28
5.1	Experiment Setup	28
5.2	A* on the Racetrack Domain	30
5.2.1	A* With Different Heuristics	31
5.2.2	A* on Both Problem Variants	36
5.3	WA* on the Racetrack Domain	39
5.3.1	WA* With Different Heuristics	39
5.3.2	WA* on Both Problem Variants	42
Chapter 6: Conclusion		45
Bibliography		47

List of Figures

Figure 2.1	Best-First Search Algorithm Pseudocode	5
Figure 3.1	Diagram of Possible Collisions in Racetrack	11
Figure 3.2	Collision detection algorithm pseudocode	12
Figure 3.3	Diagram of the Problem With Collision Detection Algorithm	13
Figure 4.1	Compute Euclidean Velocity Heuristic	21
Figure 4.2	Compute Manhattan Velocity Heuristic	22
Figure 4.3	Compute Chebyshev Velocity Heuristic	26
Figure 5.1	The Map File for den520d	29
Figure 5.2	The Map File for maze512-1-0	30
Figure 5.3	The Map File for hansen-bigger-d-wide3	30
Figure 5.4	Heuristic Pairwise Comparison for Den520d	31
Figure 5.5	Heuristic Pairwise Comparison for Maze	32
Figure 5.6	Heuristic Pairwise Comparison for Hansen	32
Figure 5.7	Number of Goal Tests for Each Heuristic in Den520d	34
Figure 5.8	Number of Goal Tests for Each Heuristic in Hansen	35
Figure 5.9	Number of Goal Tests for Each Heuristic in Maze	35
Figure 5.10	New Hampshire vs Airbag Plan Cost in Den520d	38
Figure 5.11	New Hampshire vs Airbag Number of States Generated in Den520d	38
Figure 5.12	New Hampshire vs Airbag Plan Cost in Hansen	38
Figure 5.13	New Hampshire vs Airbag Number of States Generated in Hansen	38
Figure 5.14	New Hampshire vs Airbag Plan Cost in Maze	39
Figure 5.15	New Hampshire vs Airbag Number of States Generated in Maze	39
Figure 5.16	Heuristic Comparison for Weight vs Sum of Number of States Generated in den520d	40

LIST OF FIGURES

Figure 5.17	Heuristic Comparison for Weight vs Sum of Plan Cost in den520d	40
Figure 5.18	Heuristic Comparison for Weight vs Sum of Number of States Generated in Hansen	41
Figure 5.19	Heuristic Comparison for Weight vs Sum of Plan Cost in Hansen	41
Figure 5.20	Heuristic Comparison for Weight vs Sum of Number of States Generated in Maze	42
Figure 5.21	Heuristic Comparison for Weight vs Sum of Plan Cost in Maze	42
Figure 5.22	Variant Comparison of Weight vs Sum of Number of States Generated in Den520d	43
Figure 5.23	Variant Comparison for Weight vs Sum of Plan Cost in Den520d	43
Figure 5.24	Variant Comparison for Weight vs Sum of Number of States Generated in Hansen	43
Figure 5.25	Variant Comparison for Weight vs Sum of Plan Cost in Hansen	43
Figure 5.26	Variant Comparison for Weight vs Sum of Number of States Generated in Maze	44
Figure 5.27	Variant Comparison for Weight vs Sum of Plan Cost in Maze	44

Chapter 1

Introduction

Pathfinding is a classic problem in computer science that requires an agent to find the shortest path from a starting position in a map to an ending position. This problem naturally arises from real-world uses like web mapping services and robot navigation. In a more general sense, pathfinding is a planning problem where an agent needs to find some sequence of actions that reach some goal state from a given initial state.

Planning problems, such as pathfinding, are commonly solved using heuristic search. Here, the term heuristic refers to an estimate of the cost left to reach the goal state from any other state. In a real-world map navigation system, these estimates might be computed using the Euclidean distance or some other well-defined functions.

Pathfinding is commonly used as a benchmark domain to evaluate the effectiveness of different heuristic search algorithms. Other such problems include the pancake puzzle and the sliding tile puzzle. These problems differ in properties such as heuristic accuracy, branching factor, and structure of the underlying state-space. By testing on a variety of domains, insight can be gained into the strengths and limits of these different algorithms.

Another such problem is the racetrack problem [1]. It is similar to pathfinding in that a grid of some width and height is used. However, instead of the agent only moving one grid location at a time as in standard pathfinding, the racetrack domain adds the concepts of speed and acceleration. More precisely, the agent's state is given by a map location and a current velocity. The possible actions the agent can take are then defined as an acceleration vector. This acceleration vector is summed with the velocity and position vectors to find a new location and velocity.

These new properties mean that the racetrack domain is more complicated than the grid pathfinding domain. The state-space is now larger because the velocities have to be stored. There are also more actions applicable in every state.

This thesis aims to analyze the racetrack problem to develop a better understanding of the domain in the context of heuristic search. That is, we seek to better understand the properties of the domain. Doing so clarifies its

value as a benchmark problem and allows for standardized experiments with different algorithms that can lead to new conclusions about the algorithms themselves. We make the following contributions:

1. We provide a comprehensive formulation for the problem. While several papers have used it as a benchmark, the racetrack problem is usually described informally, meaning different studies may not be entirely comparable. We study two known variants. The first is Airbag, in which the agent stops and resets its velocity when it collides with an obstacle. The second is “New Hampshire”, in which the agent cannot collide with obstacles at all [8].
2. We clearly define the Euclidean and Manhattan heuristics for this domain and an enhancement that takes the current velocity into account. We also introduce the Chebyshev heuristic, which further improves on the existing methods. In all cases, we show these heuristics are admissible and consistent, which are key properties related to using heuristic search with the A* algorithm.
3. We empirically study the difficulty of solving the problem optimally by considering the performance of A* on a variety of test problems. That is, we evaluate the performance of A* on the different variants with the different heuristics. We find that for A*, the Chebyshev heuristics generally outperform the other heuristics in the racetrack domain and that the New Hampshire variant is easier to solve.
4. We study the difficulty of the problem for suboptimal search. Such methods sacrifice solution quality for a faster search, but often are more sensitive to the accuracy of the heuristic function. We test this by experimenting with Weighted A* (WA*) using different heuristic functions. We observe that the Chebyshev heuristic’s performance degrades as the search algorithm becomes more suboptimal. We also find cases in suboptimal search where the New Hampshire variant becomes more difficult to solve.

Chapter 2

Background

In this chapter, we first formulate a general state-space search problem. We then describe Best-First Search, a general search framework that can be customized to get different heuristic search algorithms. Then, the definition for a heuristic is expanded upon by examining how they are created and what makes a heuristic effective in comparison to others. The concepts of consistent and admissible heuristics are then rigorously defined, and some key conclusions are derived from these properties, including a proof that shows consistency implies admissibility. Finally, this chapter describes the heuristic search algorithms A^* and WA^* , their optimality, and their performance in comparison to each other.

2.1 State-Space Search

Consider the grid pathfinding problem, where the goal is to find a path from a starting position to a destination. An agent that works in this environment must then solve the problem using some arbitrary algorithm that is being tested. In order for the agent to complete a search, it needs to have some representation of state. A state contains the variables that explain the agent's position within its environment. For grid pathfinding, each state would be a position, namely the x and y coordinates on the grid. Agents then use a set of unique actions to move from state to state until a sequence of operators that reach the goal is found. Once the state of the agent matches the goal state, the agent has completed the search and collected a path. We will observe this property in Figure 2.1, where BFS is defined.

The set of all states inside of a domain, denoted S , is called the state-space. In turn, the process of searching for a plan within a domain can be called a state-space search task. It can be defined as a 4-tuple that holds its constituent components. Let $\tau = (S, s_0, T, G)$ where $s_0 \in S$ represents the starting state, $T : S \rightarrow 2^S$ represents the transition function that takes in a state and generates the set of child states, and $G : S \rightarrow \{true, false\}$ represents the goal test function that marks when a state is the goal. In these problems, the transition from one state to another often has a cost.

However, the racetrack domain assumes all actions and transitions have a cost of 1. Thus, minimizing the length of a plan with A* is equivalent to minimizing the cost. Generally, a single search problem within a specific domain must have definitions for each element in the tuple τ . Chapter 3 will discuss the racetrack domain's implementation of τ .

2.2 Best-First Search

Best-first search (BFS) describes a broad class of search algorithms that are used to solve planning problems. A* and WA* are two common examples of BFS. In BFS, paths are stored implicitly through nodes, where each node corresponds to a unique state and pointer to the parent node (ie. the previous state on the path). The pseudocode for BFS is shown in Figure 2.1.

The BFS algorithm begins with s_0 and two empty sets, called *CLOSED* and *OPEN*. First, s_0 is placed in the *OPEN* set and its neighboring states, which are also added to *OPEN*, are generated using $T(s_0)$. On every step, the “best” node in *OPEN*, as defined by some evaluation function F , is selected for expansion. Node expansion is when a selected node's children are generated and processed. Define $c_k \in S$ to be one of the children of arbitrary state $s_i \in S$. We can then write $c_k \in T(s_i)$, meaning c_k is a child of s_i . The generated children of s_i are then processed as follows:

1. If c_k was already in the *OPEN* list but with a worse evaluation, then update the path that was followed to c_k by changing its parent to s_i
2. If c_k was already in the *CLOSED* list but with a worse evaluation, then update the path that was followed to c_k by changing its parent to s_i , and placing it back into *OPEN*. This is known as node re-expansion.
3. If c_k was not in either *OPEN* or *CLOSED*, it has not been expanded yet and so, generate it by adding it into *OPEN*.

Through this process of consecutively moving child nodes in and out of the open list, BFS builds a path from start to goal if one exists. Different BFS algorithms use different evaluation functions. We discuss these further in Sections 2.4 and 2.5.

2.2. BEST-FIRST SEARCH

Figure 2.1: Best-First Search Algorithm

Input: Initial state s_0
Output: Path from the initial node to a goal node or a failure message
 $CLOSED \leftarrow \emptyset$
 $OPEN \leftarrow \emptyset$
Construct initial node n_0 with $n_0.state = s_0$ and $n_0.parent = \text{null}$
Insert n_0 into $OPEN$
while *true* **do**
 if *OPEN is empty* **then**
 | **return** “no solution exists”
 else
 | $n \leftarrow$ node in $OPEN$ with the lowest cost value $F(n)$
 if $G(n)$ // *n is a goal node*
 then
 | **return** path from n_i to n
 else
 foreach $c \in T(n)$ **do**
 if *there exists* $m \in OPEN$ *such that* $m.state = c.state$ *and*
 $F(m) > F(c)$ **then**
 | $m.parent \leftarrow c.parent$
 else if *there exists* $m \in CLOSED$ *such that* $m.state = c.state$
 and $F(m) > F(c)$ **then**
 | Remove m from $CLOSED$ Add c to $OPEN$
 else if $\forall m \in (OPEN \cup CLOSED), m.state \neq c.state$ **then**
 | Add c to $OPEN$
 Remove n from $OPEN$ and place it in $CLOSED$

2.3 Heuristics

Heuristics are a fundamental component of heuristic search and are best described as distance estimates. For example, consider grid pathfinding. Here, the set of all states contains every position on the grid, and the actions include each possible movement: up, down, left and right. As the agent carries out a search algorithm, it moves between different states s until it reaches the goal t (i.e. $G(t) = \text{true}$). We define a heuristic function as $h(s) : S \rightarrow \mathbb{R} \geq 0$, a function that returns an estimate of the distance to reach the goal.

Heuristics often come from relaxations of the original problem. For example, if we take the grid pathfinding problem and ignore obstacles, a natural estimate of our distance to the goal is the Euclidean distance. Given goal state coordinates $t = (t_x, t_y)$ and state $s = (s_x, s_y)$ where $s, t \in S$, the Euclidean distance between the state and the goal state can then be calculated as:

$$h(s) = \sqrt{(t_x - s_x)^2 + (t_y - s_y)^2} \quad (2.1)$$

The Euclidean distance assumes that the agent can move diagonally. If diagonal movement is not allowed, then Manhattan distance is also a natural estimate of the distance from the goal:

$$h(s) = |t_x - s_x| + |t_y - s_y| \quad (2.2)$$

Let us now consider properties of a heuristic function. To do so, we let $h^*(s)$ be the optimal cost from s to a goal. Notice that if we are using some $h = h^*$, then h is a perfect heuristic. We now define several important properties that heuristics may have.

Definition 2.1. A heuristic function $h(s)$ is admissible if it never overestimates the distance to the goal:

$$\forall s \in S, \quad h(s) \leq h^*(s). \quad (2.3)$$

Definition 2.2. A heuristic function $h(s)$ is consistent if it never decreases from one state to another by more than the optimal distance between them. Take arbitrary states a and b , where b is a child of a . A heuristic h is then consistent if: Where $c(a, b)$ is the optimal cost between a and b .

$$h(a) \leq h(b) + c(a, b), \quad (2.4)$$

It is important to note that admissibility requires $h(t) = 0$ for any goal state t . An important property that is shown below is that a consistent

2.4. THE A* ALGORITHM

heuristic is also necessarily admissible, provided that $h(t) = 0$ for any goal state t . As we will show later in this chapter, consistent and admissible heuristics are vital for A* search. We will use this result extensively in Chapter 4 when defining our heuristic functions.

Theorem 2.3. *If h is a consistent heuristic and $h(t) = 0$ for any goal state t , then h is admissible*

Proof. Base Case: Take $t \in S$ such that t is a goal state.

Since $h(t) = 0$ by our assumption, $h(t) \leq h^*(t)$ in the base case.

Inductive Step: Assume the statement holds for any state n that is more than $d \geq 0$ steps from the goal.

Consider an optimal path from arbitrary state s to the goal.

Let s be $d + 1$ steps from the goal and let $s' \in T(s)$ be the state that is d steps from the goal.

$$h(s) \leq h(s') + c(s, s').$$

By the inductive assumption, $h(s) \leq h^*(s)$, so

$$h(s) \leq h^*(s') + c(s, s').$$

Since s' is along the optimal path from s to the goal, $h^*(s) = h^*(s') + c(s, s')$. Thus, by induction, consistency implies admissibility. $\square$

We note that this is a well-known existing result [3]. Consider two heuristic functions h_1 and h_2 . If on average, $h_1(s) \geq h_2(s)$ for some $s \in S$, then as long as h_1 is admissible, h_1 is considered a better heuristic. This is true because the higher a heuristic is without overestimating, the closer it is to $h^*(s)$. h_1 is more informed than h_2 and will give better estimates. We will later show how heuristic functions are used in BFS algorithms. In these cases, more informed heuristic functions provided solutions faster because they allow for fewer node expansions.

2.4 The A* Algorithm

The A* algorithm is an instance of BFS that uses two components as part of its evaluation function F . The first is the accumulated cost from the start state s_0 to the current state s , called $g(s)$ or the g-cost of state s . The second component is the heuristic value h . More precisely, A*'s evaluation function is the sum of $g(s)$, and the heuristic estimate of the current node.

$$F(s) = g(s) + h(s), \quad s \in S. \tag{2.5}$$

The A* algorithm is a powerful tool because it guarantees optimal solutions, provided that an admissible heuristic is used. When A* selects a node for expansion, it chooses the node with the lowest F . Since g is monotonically increasing and h is a lower bound on the remaining cost, F becomes a lower bound on the true cost of the optimal solution. If the heuristic is consistent, then there will never be a node with a better g-cost in *OPEN* for that same reason. Thus, a consistent heuristic avoids re-expansions altogether [3]. Through this process, A* will naturally select optimal nodes during its search. As a result, A* guarantees solutions with optimal cost, C^* .

2.5 Weighted A* (WA*)

Weighted A* (WA*) is a variation of A* that guarantees suboptimal solutions within a certain factor of the optimal cost. It achieves this by applying a weight $w \geq 1$ to the heuristic estimate $h(s)$ in its evaluation function.

$$F_w(s) = g(s) + w \cdot h(s) \quad s \in S, w \geq 1 \quad (2.6)$$

Its main advantage over A* is that it is usually able to find solutions with fewer node expansions, though they are no longer guaranteed to be optimal. Because WA* puts more emphasis on the heuristic estimate, it will guide itself towards the goal. This means the agent will explore less and so in general, there will be fewer node expansions and solutions will be found faster. WA* trades solution quality for speed. However, it is guaranteed to find a solution that costs no more than $w \cdot h^*(s_0)$, and thus the algorithm still has solution optimality guarantees [4].

Chapter 3

Formulating The Racetrack Problem

The previous chapter provided a background on core concepts in heuristic search. This chapter will focus on the racetrack domain in relation to heuristic search. It expands on Section 2.1 by focusing on the implementation of τ , the constituent components of the racetrack problem. This is achieved by using the grid pathfinding domain as an example. Grid pathfinding is very similar to the racetrack problem since they both require a path to be found from a start to a goal on a map. The difference here is that the racetrack domain attempts to emulate real-world races, where cars speed along a track. Thus, instead of moving through the map one tile at a time like in pathfinding, the agent moves across the map with velocity. First, the state and action representations for racetrack are defined. Then, the specific implementations of the racetrack problem components like goal state and transition system are provided. Finally, the chapter will offer more detail on the two variants of racetrack that were introduced in Chapter 1.

3.1 State

Every domain must have a representation of state to track progress along the problem. Similar to grid pathfinding where state is defined solely by the x and y coordinates of the agent, racetrack state also tracks the position of the agent along with a velocity. Velocity (v_x, v_y) is a 2-dimensional vector that represents the change in both the x and y axes. As a result, the agent is able to move through the map along a series of vectors. To allow for hashing and some useful heuristics, that will be shown in Chapter 4, the velocity in any direction is bounded by a maximum velocity v_{max} . This means $|v_x| \leq v_{max}$ and $|v_y| \leq v_{max}$. Therefore the state representation in racetrack will be the 4-tuple (x, y, v_x, v_y) where the first two elements are the x and y coordinates of the agent and the last two elements represent the current velocity vector.

3.2 Actions

To generate child nodes as in Figure 2.1, the domain must have a set of possible actions. For a given state, the transition function T will later prune out inapplicable actions to find child nodes. Since in racetrack, we only work with velocity, the actions are defined as all possible acceleration vectors (a_x, a_y) where $a_x, a_y \in \{-1, 0, 1\}$. This creates nine total actions for the domain where a_x is the acceleration in the x-axis and a_y is the acceleration in the y-axis.

3.3 Transition Function

The transition function is responsible for generating child nodes during the search. First, the transition from a state to the next must be defined. For grid pathfinding, there are only four actions. If an agent moves up or down, its y-coordinate must increase or decrease by one respectively. If an agent moves left or right, its x-coordinate must increase or decrease by one. For racetrack, since the agent has a velocity, each state transition must apply the velocity to the position, regardless of action. Consider the action $(0,0)$. Even with no change in velocity, if the agent's velocity is not zero, it will still move across the map. If the action is anything other than $(0,0)$, then the agent must move as a result of the increased velocity. Thus, we define the transition of an agent between states as:

$$x_{new} := x + v_x + a_x \quad y_{new} := y + v_y + a_y \quad (3.1)$$

The transition function must also identify the possible actions for a given state. Consider an open square map in the grid pathfinding domain. If the agent is next to the left wall, moving left should not be possible. Thus, the transition function should only generate the results of actions up, right and down. Now in a non-open map with obstacles, the transition system needs to be able to identify which of the possible actions result in a collision with an obstacle. It must then prune those actions out when generating child nodes.

The racetrack problem needs a similar system. If the agent is next to the left wall in an open map with $(0,0)$ velocity, it should not be able to move left with action $(-1, 0)$. For this scenario, it is simple enough to check whether or not an action would result in a state outside of the map. Consider a non-open map with obstacles, and an agent moving towards an obstacle with some velocity. If the agent is far enough from the obstacle to decelerate and

3.3. TRANSITION FUNCTION

not collide, there is no problem. However, the agent may select an action that would cause it to accelerate and move across the obstacle. This would have resulted in a collision that the current transition system would not have detected. We must use a collision detection algorithm to know when the agent has collided with an obstacle to prevent the agent from passing through obstacles during the search.

3.3.1 Collision Detection

Consider the following 5x5 grid with start states S_1 and S_2 at (0,1) and (4, 0) respectively. The gray tiles represent obstacles O_1, O_2 and O_3 .

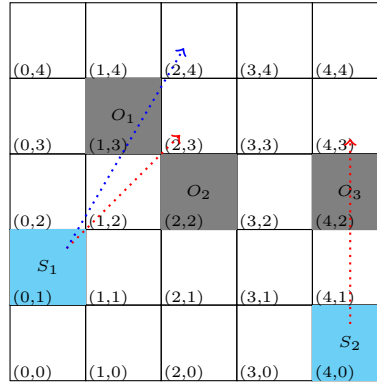


Figure 3.1

For S_2 , consider an x-velocity of 0 and a y-velocity of 3. Here, the collision would be easy to detect. We can simply trace the path of the agent from (4,0) to (4,3) since the agent only moves in the y direction. However, when the agent also moves in the x direction, the trace becomes more complicated.

For S_1 , consider an x-velocity of 2 and a y-velocity of 2; (2,2). Its important to see here that it is vague as to if the agent has crossed obstacles O_1 or O_2 since it perfectly passes through the corners in between the two. In this work, assume that the agent does not collide with obstacles in these cases.

Finally, consider S_1 with a velocity represented by (2,3), drawn with the blue arrow in the diagram above. Here, it is clear that the agent has collided with obstacle O_1 , but it remains unclear how to detect it. Which tiles did

3.3. TRANSITION FUNCTION

the agent pass on its path to its resulting tile at (2,4)? Did it pass through both (0,2) and (1,2)?

To detect these collisions, we can follow the line segment from the current location to the potential resulting location and check for collisions along the path. We do this with a series of steps along the line. First, we compute the Euclidean distance d from the current position to the resulting position. Then we compute step factor as $1/d$ so that we can calculate the step size in each axis.

$$step_x := \frac{1}{d}(v_x + a_x) \quad step_y := \frac{1}{d}(v_y + a_y) \quad (3.2)$$

This operation will allow the agent to move along the line segment in small increments and detect collisions along the way. Figure 3.2 below contains the pseudocode for the entire collision detection function.

Figure 3.2: The Collision Detection Algorithm

```

Input: Current state  $s = (x, y, v_x, v_y)$ , action  $a = (a_x, a_y)$ 
Output: If collision is detected
 $r_x \leftarrow x + v_x + a_x$ 
 $r_y \leftarrow y + v_y + a_y$ 
 $s_1 \leftarrow s$ 
 $d \leftarrow \text{round}\left(\sqrt{(r_x - x)^2 + (r_y - y)^2}\right)$ 
 $x' \leftarrow x$ 
 $y' \leftarrow y$ 
 $step\_factor \leftarrow \frac{1}{d}$ 
 $step\_size\_x \leftarrow step\_factor \cdot (v_x + a_x)$ 
 $step\_size\_y \leftarrow step\_factor \cdot (v_y + a_y)$ 
for  $i \leftarrow 1$  to  $d$  do
     $x' \leftarrow x' + step\_size\_x$ 
     $y' \leftarrow y' + step\_size\_y$ 
    if  $not\ canOccupyLocation(\text{round}(x'), \text{round}(y'))$  then
        return true
return false

```

Note that we round the values along the path to get the actual coordinates on the grid.

Problems With The Algorithm

Because this algorithm for detecting collisions rounds each x' and y' along the path, it can result in inaccurate instances. Here, the 5x5 grid that we have been using helps to demonstrate the problem. Consider the grid with updated S_1 and velocity $(1,4)$.

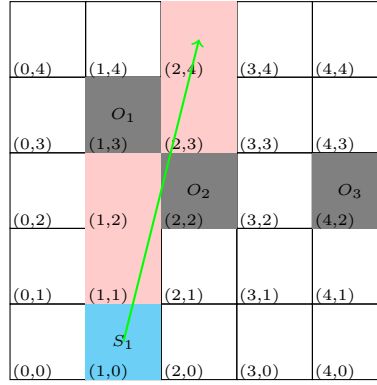


Figure 3.3

The pink shaded tiles are the tiles that the current algorithm considers. Notice that even though the agent should clearly cross O_2 , the current algorithm passes over it due to imprecise rounding. This is a small inaccuracy that diminishes the faster the agent is moving. However, in a grid with many obstacles and goal positions, the agent cannot accelerate to high velocities. This becomes a problem since smaller velocities may result in more tiles being missed and obstacles or goals being ignored. Fortunately, the race track problem domain does not often consider grids with independent tiles as goals or obstacles, since it is made to model actual racetracks. Thus, this approach is sufficient for our purposes.

3.3.2 The New Hampshire Variant

The New Hampshire variant of the racetrack domain [8] strictly prohibits actions that lead to collisions. Consider state S_2 in Figure 3.1. Suppose it had a velocity of $(0,2)$ instead of $(0,3)$. In the New Hampshire variant of the problem, only actions that decrease the y-velocity would be applicable: $v_y = -1$. Then if we applied the action $a = (0, -1)$, the next state position

would be $(4,1)$, meaning the agent does not collide with obstacle O_3 . Thus, a is an applicable action. Since we are at the right wall of the map, we also cannot apply any actions that increase v_x since this would lead to a collision with the right wall.

When an agent has too much velocity, it is impossible to select actions that slow it down enough to prevent collision. Consider S_2 in Figure 3.1 again, but this time with the original velocity of $(0,3)$. Now notice that no matter which action we choose, we cannot decrease v_y to less than 2, which means the agent is guaranteed to collide. Thus, in this state, the agent has no applicable actions and will collide. This is known as a dead end state, where the agent has no applicable actions. In general, it is best to prune out dead end states to avoid wasted time during node expansion. Since dead end states are a key feature of the New Hampshire variant, we elected to leave these states in our experiments.

3.3.3 The Airbag Variant

When dealing with collisions, we may also allow the agent to crash. Then we reset the velocity back to 0 and start again. We call this variation of the problem, the Airbag variant. Consider state S_1 in Figure 3.1 with its velocity of $(2,2)$. With this velocity, we cannot prevent it from crashing into obstacle O_1 . None of our nine actions can decelerate the agent enough. In the Airbag variant, when this collision is detected, we would set the state's position (x and y) to the last possible position on the map and reset velocity. In this case, the resulting state would be $(1, 2, 0, 0)$, representing coordinate $(1,2)$ on the grid with velocity vector $(0,0)$.

The Airbag variant does not have dead end states like the New Hampshire variant. This in theory should allow for more possible solutions than in the New Hampshire since the dead end states remain solvable in Airbag. We will observe this property in our experiments in Chapter 5.

3.4 Goal Test

Every planning problem, and by extension heuristic search problem, needs to find a goal within its working environment. A goal can either be a single state or a set of states that satisfy desired conditions to solving a problem. In grid pathfinding for example, a goal state can be a single destination on the grid. This would then need to be stored so that as a search algorithm is being run, it can check if the current expanded node is the goal state.

3.4. GOAL TEST

Since it emulates a real-world racetrack where cars complete a race by reaching the finish line, the racetrack problem environment must be able to store finish lines along the map. The simplest solution is to store a vector of states that form a line somewhere in the map. If the agent's state ever matches any of these stored states, it has reached the goal and the search is over. Thus, the goal test $G(s)$ for $s \in S$ will simply check if s matches any of the stored goal states.

In a real-world race, as long as the car crosses the finish line, the race has ended. In our implementation of the problem, we only allow the agent to land on the finish line precisely; crossing the line is not considered a valid solution. We leave consideration of this case as future work. Note that in Section 3.3.1, we discussed a collision detection algorithm that is used in the transition function. This same algorithm may be used for the goal test in the future to handle this case.

Chapter 4

Heuristics

This chapter discusses the six heuristics used to benchmark the racetrack domain. We first explain the existing heuristics that have been used with racetrack. Next, we define variants of the heuristic functions that account for velocity. Then, we will define two new heuristics derived from Chebyshev distance. Finally, we will prove consistency and admissibility for some of these heuristics. Note that as of now, we have not been able to prove consistency for every heuristic.

4.1 Existing Heuristics

The Euclidean and Manhattan distance heuristics are applicable to the racetrack problem. In racetrack, the distance estimates need to be divided by some representation of maximum velocity to maintain admissibility. When the agent is accelerating, the true number of steps would be less than the distance to the goal. Consequently, this would be an overestimating heuristic. Thus, we provide the specific adaptations commonly used for Euclidean and Manhattan distance in the racetrack problem.

Consider the states p , c and t , where $c \in T(p)$ and t is a goal. For simplicity, we use $p_x, p_y, c_x, c_y, t_x, t_y$ to represent the x, y coordinates of p , c and t . We also use v_x^p and v_y^p to denote the velocity of p , and a_x and a_y to denote the action taken in p that led to c . In the following sections, we will define various heuristics and show their consistency for a single goal. It is important to note that if we take the minimum or maximum of the heuristic computations for any state, then we will still have a consistent heuristic.

Theorem 4.1. *If $h_1(s)$ and $h_2(s)$ are consistent heuristics for all $s \in S$. Then $h_3(s) = \min(h_1(s), h_2(s))$ is also consistent.*

Proof. Let $p, c \in S$ such that $c \in T(p)$

Let $h_3 = \min(h_1, h_2)$

Note that in racetrack, $c(p, c) = 1$ for all possible parent and child nodes p and c .

4.1. EXISTING HEURISTICS

Case $h_1(p) \geq h_2(p)$:

$$h_1(p) - 1 \geq h_2(p) - 1$$

$$\implies h_1(c) \geq h_2(p) - 1 \text{ and } h_2(c) \geq h_2(p) - 1 \text{ by consistency}$$

Thus, we have

$$\min(h_1(c), h_2(c)) \geq h_2(p) - 1$$

$$\implies h_3(c) \geq h_2(p) - 1$$

Finally, since $h_3(p) = h_2(p)$ by our assumption, we have:

$$h_3(p) - h_3(c) \leq h_2(p) - h_2(p) + 1 \leq 1$$

By symmetry, this theorem holds in both cases so h_3 is consistent. $\square$

Theorem 4.2. *If $h_1(s)$ and $h_2(s)$ are consistent heuristics for all $s \in S$. Then $h_3(s) = \max(h_1(s), h_2(s))$ is also consistent.*

Proof. Let $p, c \in S$ such that $c \in T(p)$

Let $h_3 = \max(h_1, h_2)$

We know that $c(p, c) = 1$

Case $h_1(c) \geq h_2(c)$:

$$h_1(c) + 1 \geq h_1(p) \text{ and } h_1(c) + 1 \geq h_2(c) + 1 \geq h_2(p) \text{ by consistency}$$

Thus, since $h_3(c) = h_1(c)$ by our assumption, we have

$$\max(h_1(p), h_2(p)) \leq h_1(c) + 1$$

$$\implies h_3(p) \leq h_3(c) + 1$$

By symmetry, this theorem holds in both cases so h_3 is consistent. $\square$

4.1.1 Euclidean Heuristic

Recall the equation for Euclidean distance from Equation 2.1. Note that this is the same as the Euclidean norm of the goal position t subtracted by the state position s : $\|t - s\|_2$. To ensure admissibility, we need to divide by the Euclidean norm of the maximum velocity vector of $(10, 10)$. This will split the Euclidean distance into a series of steps, each as long as $\|(10, 10)\|_2$, which is the most the agent can move in one turn. Thus the Euclidean heuristic for the racetrack problem is defined as:

$$h_{\text{euc}}(s) = \frac{\|t - s\|_2}{\|(10, 10)\|_2} = \frac{\sqrt{(t_x - s_x)^2 + (t_y - s_y)^2}}{\sqrt{200}} \quad (4.1)$$

4.1. EXISTING HEURISTICS

Theorem 4.3. *The Euclidean heuristic is admissible and consistent.*

Notice that for any goal state t , $h_{euc}(t) = 0$. Now consider states p and c . We know that the following is true by the definition of h_{euc} .

$$\begin{aligned} h_{euc}(p) - h_{euc}(c) &= \frac{\sqrt{(t_x - p_x)^2 + (t_y - p_y)^2}}{\sqrt{200}} - \frac{\sqrt{(t_x - c_x)^2 + (t_y - c_y)^2}}{\sqrt{200}} \\ &= \frac{\|t - p\|_2 - \|t - c\|_2}{\sqrt{200}} = \frac{\|t - p\|_2 - \|t - (p + v + a)\|_2}{\sqrt{200}} \end{aligned}$$

Since $|v_x^p + a_x| \leq v_{max} = 10$ and $|v_y^p + a_y| \leq v_{max} = 10$ Then

$$h_{euc}(p) - h_{euc}(c) \leq \frac{\|t - p\|_2 - \|t - p - (10, 10)\|_2}{\sqrt{200}}$$

Let $k = t - p$. Then we have:

$$h_{euc}(p) - h_{euc}(c) \leq \frac{\|k\|_2 - \|k - (10, 10)\|_2}{\sqrt{200}}$$

By the triangle inequality in $\mathbb{R}^2$

$$\|k\|_2 - \|k - (10, 10)\|_2 \leq \|k - k + (10, 10)\|_2 \leq \|(10, 10)\|_2$$

Finally,

$$\frac{\|k\|_2 - \|k - (10, 10)\|_2}{\sqrt{200}} \leq 1$$

Therefore, h_{euc} is consistent for a single goal. It is also admissible by Theorem 2.1, since we showed that $h_{euc}(t) = 0$ for any goal.

From Theorem 4.1, we see that if our heuristic is consistent for a single goal state, it will be consistent for the minimum of all goals. Thus, when using this heuristic with multiple goal positions, we need to calculate the heuristic estimate of s for all goals and find the minimum. This will be true for our other heuristics as well.

4.1.2 Manhattan Heuristic

We can define the Manhattan heuristic for racetrack similarly to how we did for the Euclidean heuristic. Here, we use Equation 2.2, the formula for Manhattan distance, and divide by the Manhattan norm of the maximum velocity vector. Given a vector $a = (a_x, a_y) \in \mathbb{R}^2$, the Manhattan norm of a is defined as $\|a\|_1 = |a_x| + |a_y|$. Thus, the Manhattan heuristic for racetrack will be:

$$h_{man}(s) = \frac{|t_x - s_x| + |t_y - s_y|}{20} \quad (4.2)$$

4.1. EXISTING HEURISTICS

Theorem 4.4. *The Manhattan heuristic is admissible and consistent.*

Clearly $h_{man}(t) = 0$ for any goal state t . Now, the following is true:

$$\begin{aligned} h_{man}(p) - h_{man}(c) &= \frac{|t_x - p_x| + |t_y - p_y|}{20} - \frac{|t_x - c_x| + |t_y - c_y|}{20} \\ &= \frac{|t_x - p_x| + |t_y - p_y|}{20} - \frac{|t_x - (p_x + v_x^p + a_x)| + |t_y - (p_y + v_y^p + a_y)|}{20} \end{aligned}$$

This can be reorganized to

$$\begin{aligned} h_{man}(p) - h_{man}(c) &= \frac{|t_x - p_x| - |t_x - (p_x + v_x^p + a_x)|}{20} + \frac{|t_y - p_y| - |t_y - (p_y + v_y^p + a_y)|}{20} \\ \text{let } \Delta x &= \frac{|t_x - p_x| - |t_x - (p_x + v_x^p + a_x)|}{20} \end{aligned} \quad (4.3)$$

Let Δy be defined analogously. This means

$$h_{man}(p) - h_{man}(c) = \Delta x + \Delta y$$

Let us now consider Δx . Since $|v_x^p + a_x| \leq v_{max} = 10$, there are 2 cases to consider.

Case 1) $|t_x - p_x| \leq 10$

Here, the lowest possible value for $|t_x - (p_x + v_x^p + a_x)|$ is 0. So

$$\Delta x \leq \frac{|t_x - p_x| - 0}{20} \leq 10/20 \leq \frac{1}{2}$$

Notice the last case follows since $|t_x - p_x| \leq 10$ in this case.

Case 2) $|t_x - p_x| > 10$

Since $|v_x^p + a_x| \leq 10$ this means

$$|t_x - (p_x + v_x^p + a_x)| \geq |t_x - p_x| - 10$$

Thus,

$$\Delta x \leq \frac{|t_x - p_x| - |t_x - p_x - 10|}{20} \leq 10/20 \leq \frac{1}{2}$$

Thus in all cases, $\Delta x \leq \frac{1}{2}$. By symmetric argument, $\Delta y \leq \frac{1}{2}$. Therefore by Equation 4.3,

$$h_{man}(p) - h_{man}(c) \leq \frac{1}{2} + \frac{1}{2} \leq 1$$

Therefore, the heuristic is consistent. Since $h_{man}(t) = 0$ for any goal, this means it is admissible by Theorem 2.1. From Theorem 4.1, we know that the minimum Manhattan heuristic estimate for any state will also be consistent for multiple goals.

4.2 Velocity Heuristics

The existing heuristics assume that the agent is always moving at the maximum speed. They do not account for the time it takes the agent to accelerate. In many cases, the agent will not have enough time to build up to the maximum velocity before it reaches the goal. Thus, it would be useful to define a heuristic that accounts for the time it takes the agent to accelerate.

To account for acceleration, we can use the agent's current velocity as well as its position. That way, we can provide an estimate of how many steps the agent will take to reach the goal or pass the goal while accelerating. Assume an open map to relax the constraints of the problem. We can then compute an estimate of the number of steps to the goal by tracing the optimal straight line path from the current position of the agent, with its current velocity, to the goal. This method has been conventionally used in heuristic computations for the racetrack problem [7]. We will now cover its implementation for both the Euclidean and Manhattan heuristics specifically.

4.2.1 Euclidean Velocity Heuristic

The Euclidean velocity heuristic, $h_{euc,v}$, tracks the distance from the path, as well as the distance covered in each step with Euclidean distance. Suppose the agent starts with velocity $(0,0)$. Then in the first 10 steps, the agent's speed would increase by $(1, 1)$ each turn. Then once the agent's velocity has reached maximum velocity, we compute the distance traveled in one turn at $v = (10, 10)$ until the agent covers the entire Euclidean distance.

4.2. VELOCITY HEURISTICS

Figure 4.1: Compute Euclidean Velocity Heuristic

Input: $s = (s_x, s_y, v_x^s, v_y^s)$, $t = (t_x, t_y)$
Output: $h_{euc,v}(s)$
 $d \leftarrow \|t - s\|_2$
 $h \leftarrow 0$
 $(vx, vy) \leftarrow (|v_x^s|, |v_y^s|)$
while $d > 0$ **do**
 if $vx < v_{\max}$ **then**
 $vx \leftarrow vx + 1$
 if $vy < v_{\max}$ **then**
 $vy \leftarrow vy + 1$
 $d \leftarrow d - \sqrt{vx^2 + vy^2}$
 $h \leftarrow h + 1$
return h

Theorem 4.5. *The Euclidean velocity heuristic is consistent and admissible.*

Consider $h_{euc,v}(t)$ for any arbitrary goal state. Notice that $d = \|t - t\|_2 = 0$ and so, the final value of h would be 0. Thus, $h_{euc,v}(t) = 0$. Now, considering states p and c as well, we have 2 cases.

Case 1: $h_{euc,v}(p) = 1$

$c(p, c) = 1$ for any neighboring states p and c . Thus, we have:

$$\begin{aligned} h_{euc,v}(p) &= 1 - h_{euc,v}(c) \leq 1 \quad \text{since } h_{euc,v}(c) \geq 0. \\ \implies h_{euc,v}(p) &\leq c(p, c) + h_{euc,v}(c) \end{aligned}$$

Case 2: $h_{euc,v}(p) > 1$

We can assume that both $v_x^p \geq 0$ and $v_y^p \geq 0$ since we take their absolute values. Thus,

$$h_{euc,v}(p) = \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \|(\min(v_x^p + j, v_{\max}), \min(v_y^p + j, v_{\max}))\|_2 \geq \|p - t\|_2\}$$

We can reorganize this as:

$$\begin{aligned} h_{euc,v}(p) &= 1 + \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \|(\min(v_x^p + 1 + j, v_{\max}), \min(v_y^p + 1 + j, v_{\max}))\|_2 \\ &\geq \|p - t\|_2 - \|(v_x^p + 1, v_y^p + 1)\|_2\}. \end{aligned}$$

Since c can be at most $\|(v_x^p + 1, v_y^p + 1)\|_2$ closer to t than p , we have

$$h_{euc,v}(p) \leq 1 + \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \|(\min(v_x^p + 1 + j, v_{\max}), \min(v_y^p + 1 + j, v_{\max}))\|_2 \geq \|c - t\|_2\}$$

4.2. VELOCITY HEURISTICS

Recall that $a = (a_x, a_y)$ is the action that was applied to p leading to c . We know from Section 3.2 that $a_x \leq 1$ and $a_y \leq 1$. Therefore, the following is true:

$$h_{euc,v}(p) \leq 1 + \min \left\{ i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \left\| (\min(v_x^p + a_x + j, v_{\max}), \min(v_y^p + a_y + j, v_{\max})) \right\|_2 \geq \|c - t\|_2 \right\}.$$

Observe that the second term on the right-hand side above is just $h_{euc,v}(c)$. Thus,

$$h_{euc,v}(p) \leq 1 + h_{euc,v}(c)$$

Therefore, $h_{euc,v}$ is a consistent heuristic. Since $h_{euc,v}(t) = 0$ for any goal t , it is also admissible by Theorem 2.1. By Theorem 4.1, if we choose the minimum $h_{euc,v}$ between all goals, then the heuristic will also be consistent for multiple goals as well.

4.2.2 Manhattan Velocity Heuristic

Recall, from Equation 2.2, the Manhattan distance for any two points on the map. We can then formulate the Manhattan velocity heuristic similarly to how we did the Euclidean velocity heuristic, but with Manhattan distance instead. Let $h_{man,v}$ be the Manhattan velocity heuristic function. Assuming d is the starting Manhattan distance of the agent from the goal, the number of steps it takes the agent to cross this distance will be our heuristic estimate.

Figure 4.2: Compute Manhattan Velocity Heuristic

Input: $s = (s_x, s_y, v_x^s, v_y^s)$, $t = (t_x, t_y)$

Output: $h_{man,v}(s)$

$d \leftarrow \|t - s\|_1$

$h \leftarrow 0$

$(vx, vy) \leftarrow (|v_x^s|, |v_y^s|)$

while $d > 0$ **do**

if $vx < v_{\max}$ **then**

$vx \leftarrow vx + 1$

if $vy < v_{\max}$ **then**

$vy \leftarrow vy + 1$

$d \leftarrow d - |vx + vy|$

$h \leftarrow h + 1$

return h

4.2. VELOCITY HEURISTICS

Theorem 4.6. *The Manhattan velocity heuristic is consistent and admissible.*

The proof for this theorem follows the same logic as for Theorem 4.5. Consider $h_{man,v}(t)$ for any arbitrary goal state. Similar to the Euclidean velocity heuristic, since $d = |t - t| = 0$, $h_{man,v}(t) = 0$. We again consider states p and c in the following cases.

Case 1: $h_{man,v}(p) = 1$

$$c(p, c) + h_{man,v}(c) = 1 + h_{man,v}(c) \geq 1 \text{ since } h_{man,v} \geq 0.$$

Thus, we have $1 + h_{man,v}(c) \geq h_{man,c}(p)$

Case 2: $h_{man,v}(p) > 1$

Again, we assume that both $v_x^p \geq 0$ and $v_y^p \geq 0$ since we take their absolute values here as well. Thus,

$$h_{man,v}(p) = \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \|(\min(v_x^p + j, v_{max}), \min(v_y^p + j, v_{max}))\|_1 \geq \|p-t\|_1\}$$

$$\begin{aligned} h_{man,v}(p) &= 1 + \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \|(\min(v_x^p + 1 + j, v_{max}), \min(v_y^p + 1 + j, v_{max}))\|_1 \\ &\geq \|p-t\|_1 - \|(v_x^p + 1, v_y^p + 1)\|_1\}. \end{aligned}$$

$$h_{man,v}(p) \leq 1 + \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \|(\min(v_x^p + 1 + j, v_{max}), \min(v_y^p + 1 + j, v_{max}))\|_1 \geq \|c-t\|_1\}$$

$$\begin{aligned} h_{man,v}(p) &\leq 1 + \min\left\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \left\| \left(\min(v_x^p + a_x + j, v_{max}), \min(v_y^p + a_y + j, v_{max}) \right) \right\|_1 \right. \\ &\quad \left. \geq \|c-t\|_1 \right\}. \end{aligned}$$

$$h_{man,v}(p) \leq 1 + h_{man,v}(c)$$

These steps above are analogous to the proof for the consistency of the Euclidean velocity heuristic in Section 4.2.1. Thus, the Manhattan Velocity heuristic is consistent and admissible since $h_{man,v}(t) = 0$ for any goal t . By Theorem 4.1, this result also generalizes to multiple goals if the minimum of these estimates is taken.

4.3 New Heuristics Based on Chebyshev Distance

We now introduce two new heuristics for the racetrack domain based on Chebyshev distance. Given a goal state t and a state s , the Chebyshev distance from s to t is defined as:

$$d_{Chebyshev} = \max(|t_x - s_x|, |t_y - s_y|) \quad (4.4)$$

We will define two heuristics, using Chebyshev distance, that mirror our existing position-based and velocity-based heuristics. In Chapter 5, we will observe the performance benefits of both Chebyshev heuristics. We also provide some details on which properties of the Chebyshev heuristic cause its performance gains.

4.3.1 Chebyshev Heuristic

As we covered in Section 4.1, to make heuristics admissible in the racetrack domain, we need to divide distance estimates by the maximum possible velocity. As such, given goal state t and starting state s , we can define the Chebyshev heuristic as:

$$h_{cheb}(s) = \frac{\max(|t_x - s_x|, |t_y - s_y|)}{10} \quad (4.5)$$

Theorem 4.7. *The Chebyshev heuristic is consistent and admissible.*

Again, notice that $h_{cheb}(t) = 0$ for any goal state t . Now, with states p and c , we know the following:

$$\begin{aligned} h_{cheb}(p) - h_{cheb}(c) &= \frac{\max(|t_x - p_x|, |t_y - p_y|)}{10} - \frac{\max(|t_x - c_x|, |t_y - c_y|)}{10} \\ &= \frac{\max(|t_x - p_x|, |t_y - p_y|)}{10} - \frac{\max(|t_x - (p_x + v_x^p + a_x)|, |t_y - (p_y + v_y^p + a_y)|)}{10}. \end{aligned}$$

From Section 3.1, we know that $|v_x^p + a_x| \leq 10$ and $|v_y^p + a_y| \leq 10$. Thus the following inequality is true:

$$h_{cheb}(p) - h_{cheb}(c) \leq h_{cheb}(p) - \frac{\max(|t_x - p_x - 10|, |t_y - p_y - 10|)}{10}$$

Since h_{cheb} has two possible values, there are two cases to consider. However, since the argument is the same for both cases, we will use a symmetric

argument.

Suppose $h_{\text{cheb}}(p) = \frac{t_x - p_x}{10}$. Then we have

$$t_x - p_x > t_y - p_y \implies t_x - p_x - 10 > t_y - p_y - 10$$

Then

$$h_{\text{cheb}}(p) - h_{\text{cheb}}(c) \leq \frac{t_x - p_x - t_x + p_x + 10}{10} = 1$$

By a symmetric argument, the above also holds for if $h_{\text{cheb}}(p) = \frac{t_y - p_y}{10}$. Thus, h_{cheb} is consistent and since $h_{\text{cheb}}(t) = 0$ for all goal states t , it is also admissible by Theorem 2.1. By Theorem 4.1, we also know that the minimum estimate of h_{cheb} is consistent for multiple goal states.

4.3.2 Chebyshev Velocity Heuristic

Similar to the Manhattan and Euclidean heuristics, it makes sense to define a variation of the Chebyshev heuristic that accounts for velocity. Let $h_{\text{cheb},v}$ be the Chebyshev velocity heuristic. Since Chebyshev distance takes the maximum of both the x and y differences between state s and goal t , we can replicate this property in Chebyshev velocity by computing two estimates for each direction. We then choose the maximum of these two estimates as our overall heuristic estimate. This will also allow us to provide an estimate of the time it takes the agent to turn around if its velocity was headed away from the goal.

Suppose we are computing the Chebyshev velocity heuristic estimate for s in the x direction. Similar to the other two velocity heuristics, we could increase the velocity of s and use Chebyshev distance to track the agent's progress along the distance. Now suppose we were doing this computation in the Airbag variant. Assume the current state is (10, 10, -10, 0) and the goal is at (11, 10). Here, the velocity of the agent is headed away from the goal. We would then take several steps to turn the agent around, before we can begin to approach the goal. Now suppose there is a wall at (9, 10). In the airbag variant, the agent would crash into the wall, reset its velocity to 0, and immediately approach the goal. The true cost here would be 2, but our heuristic estimate would be much higher because it would not detect the collision. Consequently, this approach in Airbag is inadmissible.

To resolve the issue, the Chebyshev velocity function should handle the Airbag environment differently. In Airbag, we should assume a collision will occur if the agent's velocity is headed away from the goal. Thus, we set velocity to 0 and increment the heuristic estimate by one, imitating a crash.

4.3. NEW HEURISTICS BASED ON CHEBYSHEV DISTANCE

Below is the pseudocode for the Chebyshev velocity heuristic function.

Figure 4.3: Compute Chebyshev Velocity Heuristic

Input: $s = (s_x, s_y, v_x^s, v_y^s)$, $t = (t_x, t_y)$, track_type
Output: $h_{cheb,v}(s)$
Function CountStepsToGoal(d, v):

```

     $h \leftarrow 0$ ;
     $a \leftarrow 1$ ;
    if  $d < 0$  then
         $d \leftarrow -d$ ;
         $v \leftarrow -v$ ;
    if track_type = AIRBAG and  $d > 0$  and  $v < 0$  then
         $v \leftarrow 0$ ;
         $h \leftarrow h + 1$ ;
    while  $d > 0$  do
        if  $|v| \leq |v_{\max}|$  or  $v = -v_{\max}$  then
             $v \leftarrow v + a$ ;
        else
             $v \leftarrow v_{\max}$ ;
             $d \leftarrow d - v$ ;
             $h \leftarrow h + 1$ ;
    return  $h$ ;
 $h_x \leftarrow \text{CountStepsToGoal}(t_x - s_x, v_x)$ ;
 $h_y \leftarrow \text{CountStepsToGoal}(t_y - s_y, v_y)$ ;
return  $\max(h_x, h_y)$ ;

```

Theorem 4.8. *The Chebyshev velocity heuristic is consistent and admissible.*

To prove this theorem, we will again use parent and child states p and c , as well as goal state t . First, consider $h_{cheb,v}(t)$. From the pseudocode above, we can see that $h_{cheb}(t) = 0$ since d will be 0. Now, like in our proofs for Theorems 4.5 and 4.6, we again consider two cases.

Case $h_{cheb,v}(p) = 1$:

$$h_{cheb,v}(p) = 1 \leq c(p, c) + h_{cheb,v}(c)$$

since $c(p, c) = 1$ and $h_{cheb,v}(c) \geq 0$.

Case $h_{cheb,v}(p) > 1$:

Here, we have two sub-cases to consider. Since the CountStepsToGoal is the same for both x and y directions, by symmetry, the following arguments

4.3. NEW HEURISTICS BASED ON CHEBYSHEV DISTANCE

for the x direction apply to the y direction as well.

Sub-case 1: $v_x(p)$ does not have the same sign as $t_x - p_x$ and the variant is Airbag:

$$h_{cheb,v}(p) = \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \min(v_x^p + j, 10) \geq (t_x - p_x)\} + 1$$

We added one because in this case in Airbag, $h_{cheb,v}$ will set velocity to 0 and increment the heuristic estimate by 1. Then, we can reorganize and follow similar steps to the proofs of Theorems 4.5 and 4.6:

$$h_{cheb,v}(p) = 1 + \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \min(v_x^p + 1 + j, 10) \geq (t_x - p_x) - (v_x^p + 1)\} + 1$$

$$h_{cheb,v}(p) \leq 1 + \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \min(v_x^p + 1 + j, 10) \geq t_x - c_x\} + 1$$

$$h_{cheb,v}(p) \leq 1 + \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \min(v_x^p + a_x + j, 10) \geq t_x - c_x\} + 1$$

$$h_{cheb,v}(p) \leq 1 + h_{cheb,v}(c)$$

Sub-case 2: $v_x(p)$ has the same sign as $t_x - p_x$ or the variant is New Hampshire.

In this case, regardless of the racetrack variant, $h_{cheb,v}$ handles the calculation in the same way. Thus, the same steps above would be applied except that $h_{cheb,v}(p)$ would not add an extra one to account for collisions.

$$h_{cheb,v}(p) = \min\{i \in \mathbb{Z}^{\geq 0} \mid \sum_{j=1}^i \min(v_x^p + j, 10) \geq (t_x - p_x)\}$$

By symmetry, the same argument applies to the heuristic estimates for both axes. Since both directional estimates are consistent, and from Theorem 4.2 we know that the maximum of two consistent heuristics is also consistent, then $h_{cheb,v}$ is consistent. By Theorem 2.1, since $h_{cheb,v}(t) = 0$, then Chebyshev velocity is also admissible. Finally, from Theorem 4.1, we know that if we take the minimum value of $h_{cheb,v}(s)$, for any state s , then that heuristic estimate will also be consistent for multiple goals.

Chapter 5

Experiments

In this chapter, we discuss the experiments that we conducted on the racetrack problem. We will first discuss the setups for the experiment, the maps that were used, as well as how each problem was initialized. We will then analyze the results of both A* and WA* on the problem domain. We will show the results of the heuristics, as well as the measured empirical differences for each environment. Finally, we will discuss the impacts that weight had on the performance of our heuristics with WA*.

5.1 Experiment Setup

For this work, the racetrack problem was implemented in the Heuristic Search Experimental Framework (HSEF), created by Dr. Rick Valenzano. HSEF is a C++ library with implementations for A* and WA*. The race-track environment for HSEF was developed with the implementation details outlined in Chapter 4.

For our experiments, we used three different maps: den520d from the game Dragon Age: Origins [5], maze512-1-0, a randomly generated 512×512 maze [6], and hansen-bigger-d-wide3 [2]. The first two maps are publicly available at Nathan Sturtevant’s movingai.com, and hansen is available on Tianyi Gu’s github page, github.com/gtianyi/searchDomainInstanceFiles. 50 valid starting positions were randomly generated for each map. We also manually selected 1 finish line for hansen and 5 finish lines for den520d and the maze. By using every combination of start state and finish line, this gives us a total of $2(50 \times 5) + 50 = 550$ problems.

We manually selected the following 5 finish lines for den520d:

1. (67, 223) to (157, 223)
2. (287, 213) to (241, 213)
3. (6, 214) to (20, 214)
4. (224, 126) to (241, 126)

5. (149, 187)

These goals are spread around the map to get a good measure of performance. For the maze, because it is naturally a more difficult map than den520d, we made the goal lines much larger:

1. (199, 511) to (222, 511)
2. (511, 403) to (511, 412)
3. (0, 1) to (511, 1), along the upper wall of the map
4. (0, 511) to (511, 511), along the bottom wall of the map
5. (511, 0) to (511, 511), along the lower wall of the map

Finally, since the hansen map is much smaller and straightforward, being a traditional racetrack map rather than a grid pathfinding map, we only used one goal line from (394, 65) to (413, 65). This would be the bottom right wall in hansen.



Figure 5.1: The map file for den520d

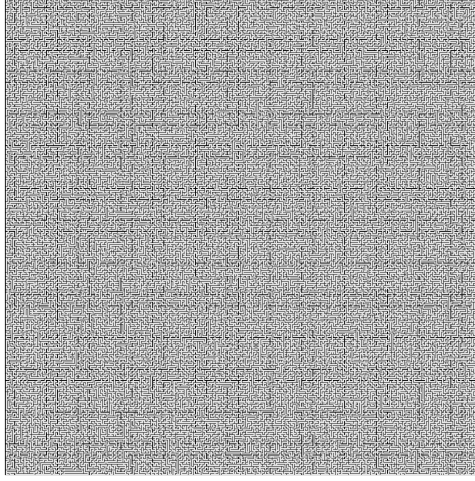


Figure 5.2: The map file for maze512-1-0



Figure 5.3: The map file for hansen-bigger-d-wide3

All of these 550 problems were run in both variants of racetrack, with both high-g and low-g tie-breaking, as well as each of the 6 heuristics covered in Chapter 4. All experiments were conducted on a 2021 MacBook Pro with the M1 Pro CPU and 16 GB of memory. From Figure 2.1, we know that a goal test and a state generation happen at every iteration of BFS. Thus, we used the number of goal tests and number of states generated to measure runtime.

5.2 A* on the Racetrack Domain

In this section, we will discuss the results of the 13,200 experiments run with A* on each map. To provide analysis on the racetrack domain, we first need to understand the maps better. The average number of search steps across all experiments for den520d, hansen and maze were 735,972.9, 915,274.1, and 351,153.4 respectively. The average cost of a plan for each

5.2. A* ON THE RACETRACK DOMAIN

map was 23.718, 94.57, and 1042.33, in the same order. Thus, from our experiments, we observed that the maze had the fewest number of states generated, but the highest plan costs. Intuitively, this is because the maze was more closed off than the other two maps.

Since the maze has thin channels with a width of 1 unit, and diagonal movements are mostly prohibited due to collisions, there is little opportunity for the agent to take a variety of actions. It will mostly move forwards or backwards along the maze. Consequently, there are fewer possible states to generate that do not lead to the optimal plan. Thus, the agent has fewer opportunities to explore, lowering the cost of the search.

Hansen as a middle ground between den520d and the maze, had the largest number of search steps and the second largest average plan costs. The hansen map is less open than den520d. Because of the many walls, the agent has to slow down and speed up multiple times. This will extend the plan cost on average. The agent is also able to explore a lot because the map is still fairly open. Thus, hansen will also have more search steps than maze but fewer than den520d.

5.2.1 A* With Different Heuristics

Our first results show the comparisons between each heuristic for number of goal tests. We observe from Figures 5.4, 5.5 and 5.6 the results for the each map respectively.

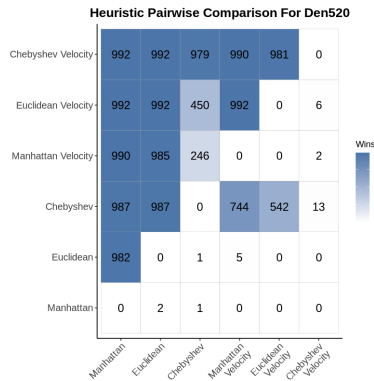


Figure 5.4: Runtime pairwise comparison for each of the heuristics used in the den520d map

5.2. A* ON THE RACETRACK DOMAIN

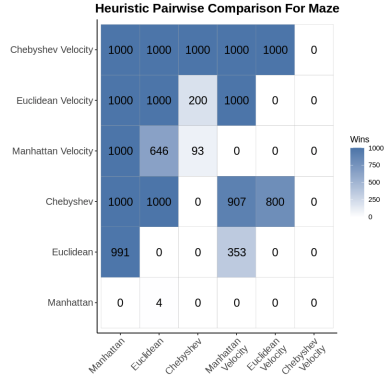


Figure 5.5: Runtime pairwise comparison for each of the heuristics used in the maze

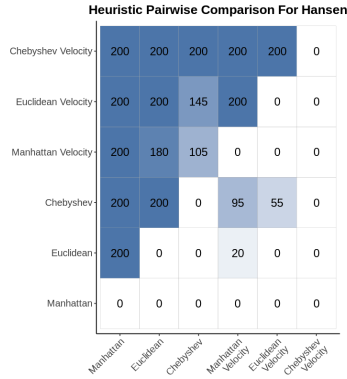


Figure 5.6: Runtime pairwise comparison for each of the heuristics used in the hansen map

In these matrices, each cell represents the number of experiments in which the row heuristic outperformed the column heuristic, with a lower number of goal tests. There are 1000 problems for each heuristic, and ties are not counted inside the matrix. Observe that for both the maze and den520d, the Chebyshev velocity and Chebyshev heuristics outperformed all other heuristics. In general, the velocity heuristics outperformed the non-velocity heuristics. Both variations of the Manhattan heuristic performed the worst. For the hansen map, we can see that Chebyshev velocity still performs the best, but now the regular Chebyshev heuristic no longer outperforms the Euclidean and Manhattan velocity heuristics.

5.2. A* ON THE RACETRACK DOMAIN

Consider Equations 4.1, 4.2 and 4.5 for the Euclidean, Manhattan and Chebyshev heuristics respectively. Suppose we had start state $s = (0,0)$ and goal state $t = (3,5)$. The computed heuristic values for Euclidean, Chebyshev and Manhattan heuristics would be

$$h_{euc}(s) = \frac{\sqrt{3^2 + 5^2}}{\sqrt{200}} \approx 0.41 \quad h_{cheb}(s) = \frac{5}{10} \quad h_{man}(s) = \frac{8}{20}$$

Notice that the denominators for all heuristic functions are constant. Thus, the Chebyshev heuristic will have the smallest denominator out of all three, and Manhattan will have the largest. Consequently, the Chebyshev heuristic will most often be the largest, with Euclidean and Manhattan following in that order. Thus, Chebyshev is generally more informed than the other two, which explains the performance observed in the figures above.

For the velocity heuristics, we observe similar properties. Recall from Section 4.2 that the velocity heuristics compute their estimates by iteratively covering a distance left using their own distance estimate. In the case of the Euclidean velocity heuristic for example, it computes the distance to the goal with Euclidean distance. Then, it counts the number of turns needed to accelerate velocity and cover that distance. The distance covered at each iteration is also computed using Euclidean distance. Once again, suppose we had a start state $s = (0, 0)$ with velocity $v = (0,0)$. We then have the following iterations on distance covered for each velocity heuristic function:

$$h_{Euclidean} : \sqrt{2}, 2\sqrt{2}, 3\sqrt{2} \approx 4.24$$

$$h_{Manhattan} : 2, 4, 6$$

$$h_{Chebyshev} : 1, 2, 3$$

Thus, after three iterations, Manhattan covers the most distance and Chebyshev covers the least. This is the prime reason why the Chebyshev velocity heuristic outperforms Manhattan and Euclidean velocity heuristics. Since it covers less distance over time, there will be more cases where the heuristic estimate for Chebyshev velocity will need more iterations to cover the whole distance and so, it will be higher. This is the same reason why Euclidean velocity is more informed than Manhattan velocity. Another distinction is between the velocity heuristics like Euclidean velocity, and the non-velocity heuristics like Euclidean. The velocity heuristics account for the steps it takes to accelerate, whereas the non-velocity heuristics assume constant maximum velocity. Thus, the velocity heuristics are an upper bound

5.2. A* ON THE RACETRACK DOMAIN

for their non-velocity counterparts, meaning that they are more informed heuristics.

Now we will discuss how much of a difference we observed between the heuristics with Figures 5.7 - 5.9. These graphs compare the frequency distribution, in terms of number of states generated, for each heuristic. Each goal test count shows the number of problems that required no more than that count of tests. Figure 5.7 shows the frequency of problems with a range of goal tests for the den520d map. Notice that Chebyshev velocity has the largest number of problems with the lowest number of goal tests. This graph is again indicating which heuristics are the most informed. We can also see the properties of the maps from these graphs.

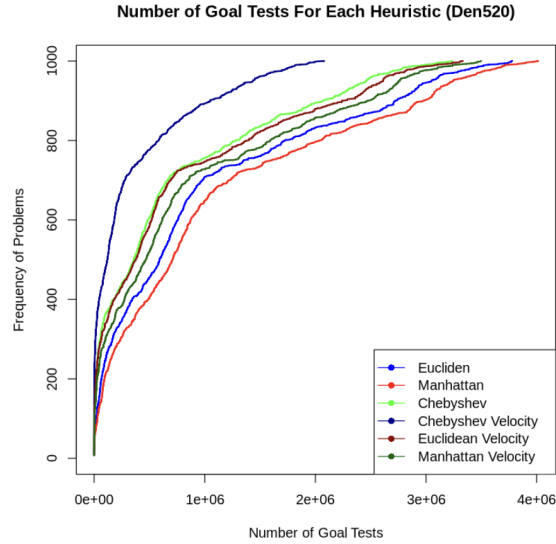


Figure 5.7: Frequency distribution of problems by number of goal tests for den520d

5.2. A^* ON THE RACETRACK DOMAIN

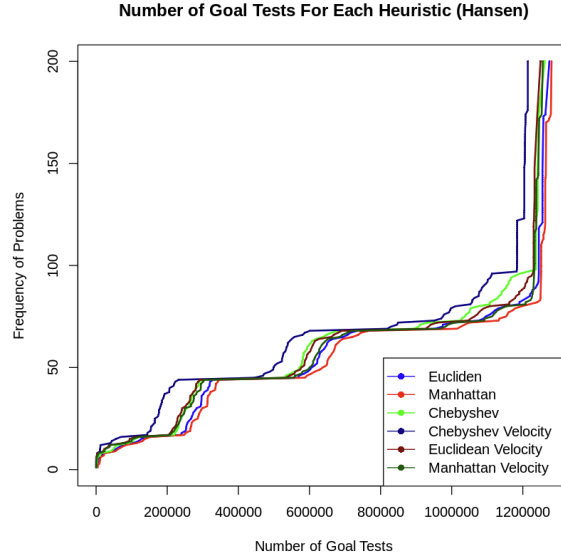


Figure 5.8: Frequency distribution of problems by number of goal tests for den520d hansen

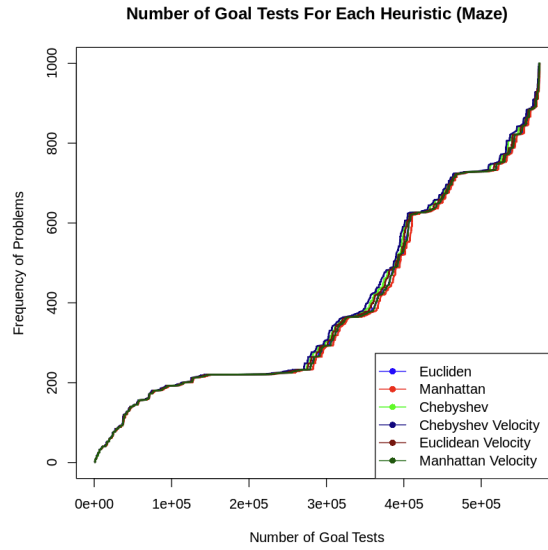


Figure 5.9: Frequency distribution of problems by number of goal tests for maze

Since the den520d map is the most open map, we know that it has room for the largest number of node expansions. Consequently in this map, the agent will have more states to explore, meaning its heuristic estimate becomes more valuable. In hansen, where there is still some room for exploration, most paths are closed off to follow the trail that we observe in Figure 5.3. So although there is room for node expansion, there will not be nearly as many as in den520d. Finally, inside the maze, we will have the fewest number of expansions for alternate paths since the maze does not allow for nearly as many alternate paths as the other two maps.

We observe these properties in the figures above. Notice that in Figure 5.7, the difference between the heuristics is the greatest. Since there are more states to explore, there are more alternate paths, and a more informed heuristic will help guide the agent through those paths. In maps where there are fewer alternate paths, informed heuristics will become less important since the agent has fewer decisions on which node to expand. Thus, we observe from Figure 5.8 that the difference between the heuristics is less pronounced, and in Figure 5.9, the heuristics have a very similar number of goal tests.

5.2.2 A* on Both Problem Variants

Recall that the New Hampshire variant of the problem does not allow the agent to generate actions that would lead to collisions. This behavior leads to two main differences in performance.

First of all, since the New Hampshire variant has more restrictions than Airbag, any viable plan in New Hampshire would be viable in the Airbag domain. However, not all plans in Airbag will be possible in New Hampshire. We can also conclude that since Airbag allows for collisions, its optimal plans must be faster than those in New Hampshire since in the latter domain, the agent needs extra actions to slow down. We observe this property in Figures 5.10, 5.12, and 5.14 for the den520d, hansen and maze maps respectively. These graphs plot the Airbag plan costs against the New Hampshire plan costs for each problem. Any point above the line of $y = x$ indicates a higher plan cost for New Hampshire. Notice that no points are below the line. Its also important to note that since den520d is the most open map, the one with the fewest collisions, many of the problems have identical optimal solutions in both variants. Hansen has more collisions so there are fewer similar solutions shared between the two variants. In all three maps, we see the property that optimal Airbag plan costs serve as a lower bound for New Hampshire plan costs.

The number of collisions will also affect the number of states generated. Since in Airbag, there are no restrictions on which actions are applicable, there are more branches to solutions that will be explored. This will naturally lead to more possible solutions and thus more states generated. We see this property in Figures 5.11, 5.13 and 5.15. These graphs plot the Airbag number of states generated against the New Hampshire number of states generated for each problem. Points below the line of $y = x$ indicate a problem that required more states generated in Airbag than in New Hampshire. The den520d experiences the smallest difference between Airbag and New Hampshire number of states generated because it experiences the fewest collisions. Hansen experiences a larger difference in number of states generated, and the maze experiences a massive difference, where Airbag has increasingly more states generated than New Hampshire.

5.2. A^* ON THE RACETRACK DOMAIN

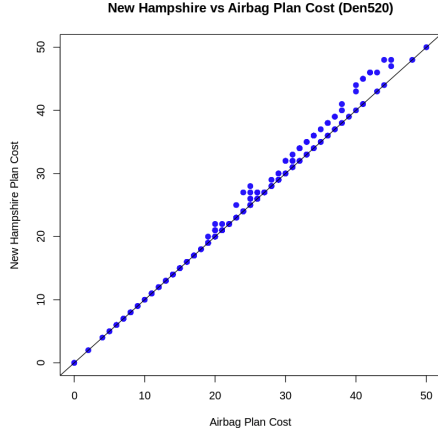


Figure 5.10: One-to-one scatter plot to compare plan costs in Airbag and New Hampshire in den520d

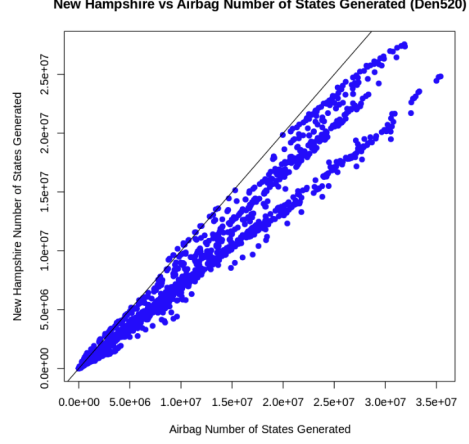


Figure 5.11: One-to-one scatter plot to compare run time for Airbag and New Hampshire in den520d

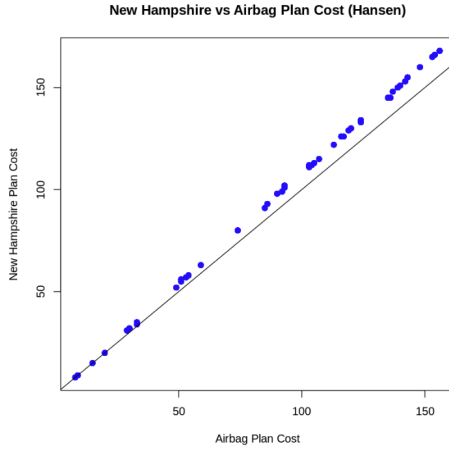


Figure 5.12: One-to-one scatter plot to compare plan costs in Airbag and New Hampshire in hansen

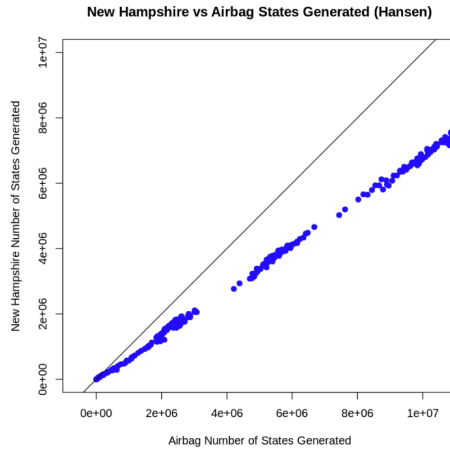


Figure 5.13: One-to-one scatter plot to compare run time for Airbag and New Hampshire in hansen

5.3. WA* ON THE RACETRACK DOMAIN

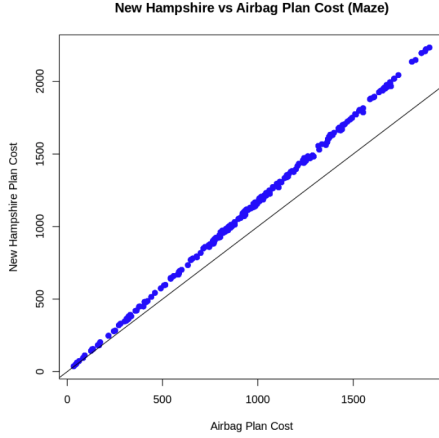


Figure 5.14: One-to-one scatter plot to compare plan costs in Airbag and New Hampshire in maze

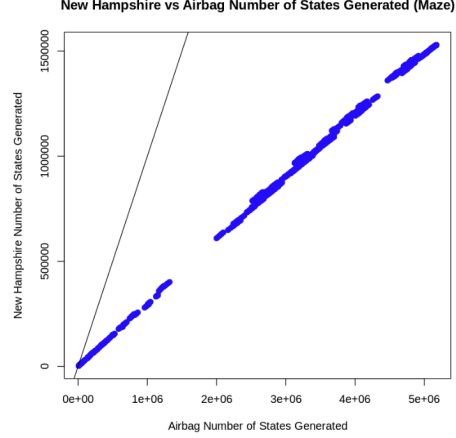


Figure 5.15: One-to-one scatter plot to compare run time for Airbag and New Hampshire in maze

5.3 WA* on the Racetrack Domain

We now compare the performance of the WA* algorithm across our test-bed. We ran each of our 13,200 problems in WA* with weights ranging from 1 to 10. This led to 132,000 total problems for WA*. In our charts, we plot the performance results for each of the tests against the weight that WA* was run with. Notice that when $w = 1$, WA* is the same algorithm as A*. Thus, we will also compare the difference in performance between A* and WA*. In general, we observe that as weight increases, solution quality decreases and number of states generated decreases. This is an expected result that we noted in Section 2.5.

5.3.1 WA* With Different Heuristics

For den520d's number of states generated, Manhattan and Manhattan velocity heuristics still generate the most states and Manhattan velocity's performance in relation to the other heuristics degrades as weight increases. At the highest weight of 10, Manhattan and Manhattan velocity perform the worst. We notice that the Chebyshev velocity heuristic no longer outperforms Euclidean velocity and Chebyshev as weight increases. In fact, as weight increases, Chebyshev velocity and Chebyshev converge to the same

5.3. WA* ON THE RACETRACK DOMAIN

number of states generated. Euclidean velocity on the other hand, has the fewest number of states generated as weight increases.

In terms of solution quality, Chebyshev is consistently worse than the other heuristics for WA*. Conversely, the Chebyshev velocity heuristic's solution quality suffers the least as weight increases compared to the other heuristics. Manhattan does the best at lower weights greater than 1, and at weights past 4, is the second best heuristic in terms of solution quality. Euclidean and Euclidean velocity are relatively tied in performance, however, Euclidean does outperform Euclidean velocity at higher weights of 9 and 10, as well as lower weights of 2 and 3. Finally, as weight increases, Manhattan velocity's solution quality becomes the second worst after a weight of 6.

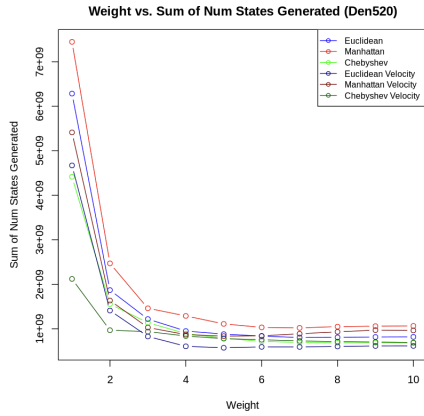


Figure 5.16: Heuristic Comparison for Weight vs Sum of Number of States Generated in Den520d

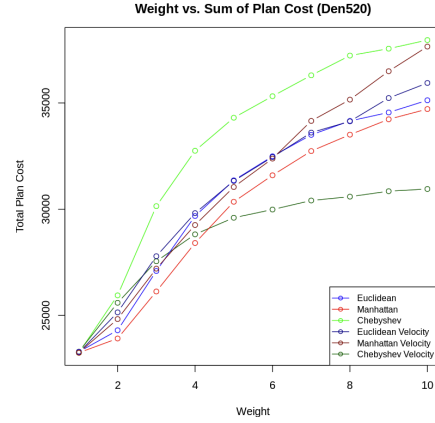


Figure 5.17: Heuristic Comparison for Weight vs Sum of Plan Cost in Den520d

Figures 5.18 and 5.19 describe the performance of the hansen map for different weights in WA*. The Manhattan and Manhattan velocity heuristics still generate the most states in WA*, but interestingly, at a weight of 10, Manhattan velocity outperforms the Euclidean heuristic. Like in den520d, Euclidean velocity eventually outperforms the Chebyshev and Chebyshev velocity heuristics, however, it happens at a later weight of 8. Chebyshev velocity also experiences a brief spike at a weight of 3 and 4 but later falls below the other heuristics in terms of generated states. Chebyshev and Chebyshev velocity also seem to converge to the same number of generated states as weight increases.

In terms of plan cost, we again see a general increase with higher weights,

5.3. WA* ON THE RACETRACK DOMAIN

but now, Euclidean, Euclidean velocity, and Manhattan outperform Chebyshev velocity as weight increases. Chebyshev still performs the worst, followed by Manhattan Velocity.

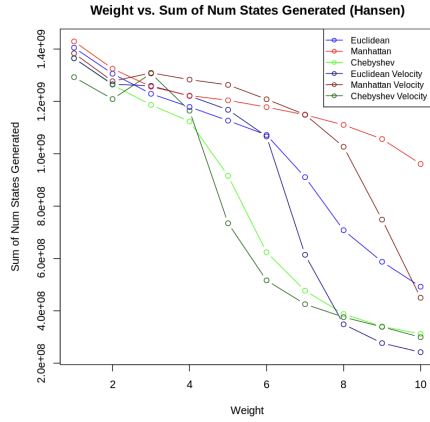


Figure 5.18: Heuristic comparison for weight vs sum of number of states generated in hansen

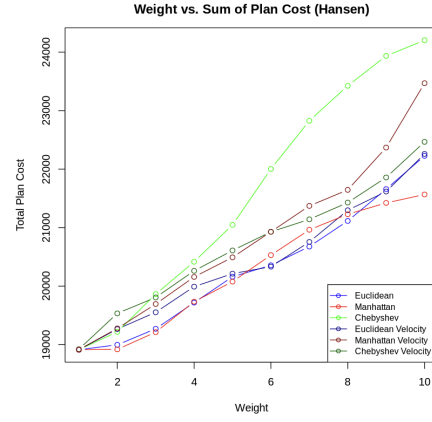


Figure 5.19: Heuristic comparison for weight vs sum of plan cost in hansen

Finally, in the maze, the number of states generated decreases almost linearly as weight increases. Chebyshev is the best heuristic here, after a weight of 3. Chebyshev velocity spikes above Chebyshev at this weight, and then proceeds to decrease. After the spike, it is the second worst heuristic. It eventually surpasses Euclidean velocity at weight 4, Manhattan at weight 5, and Euclidean after weight 6, to become the second best heuristic in terms of number of states generated.

In terms of plan cost, here we see that the velocity heuristics are clearly being outperformed by the non-velocity counterparts. This is possibly because the velocity heuristics account for acceleration towards the goal, but the maze does not allow for much acceleration. Here Manhattan does the best, with the lowest plan cost throughout the different weights, and Euclidean velocity does the worst for weights 2 and anything past 5, while Chebyshev does the worst for weights 3 and 4.

5.3. WA* ON THE RACETRACK DOMAIN

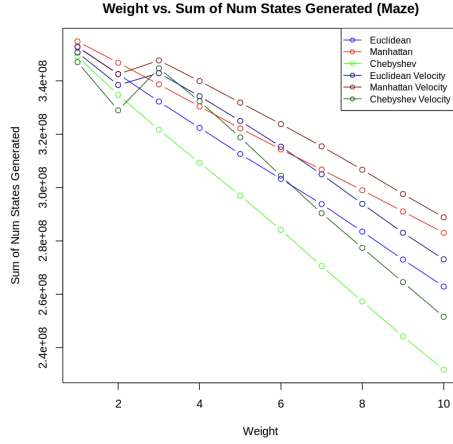


Figure 5.20: Heuristic comparison for weight vs sum of number of states generated in maze

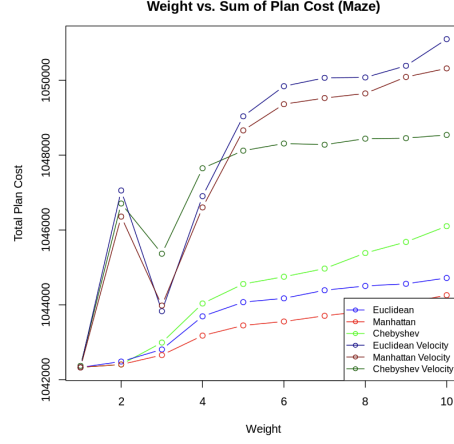


Figure 5.21: Heuristic comparison for weight vs sum of plan cost in maze

5.3.2 WA* on Both Problem Variants

We will now analyze the performance of WA* with different weights on the two variants of racetrack. We can clearly see in Figure 5.26 that New Hampshire generates fewer states than Airbag in den520d. This property carries over from A* for the same reasons we explained in Section 5.2.2. WA* also seems to have the greatest decrease in number of states generated moving from weight 1 to 2. At subsequent weights, the performance benefits decrease. For plan cost, we notice from Figure 5.23 that in the den520d map, New Hampshire actually experiences better solution quality for weights higher than 3.

5.3. WA* ON THE RACETRACK DOMAIN

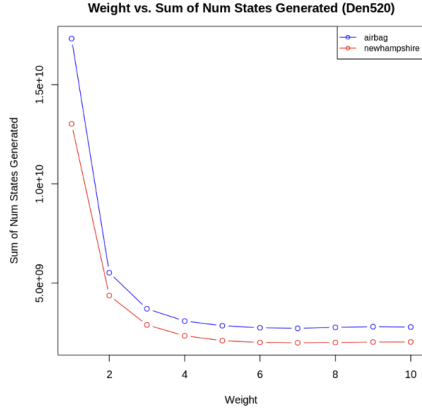


Figure 5.22: Variant comparison of weight vs sum of number of states generated in den520d

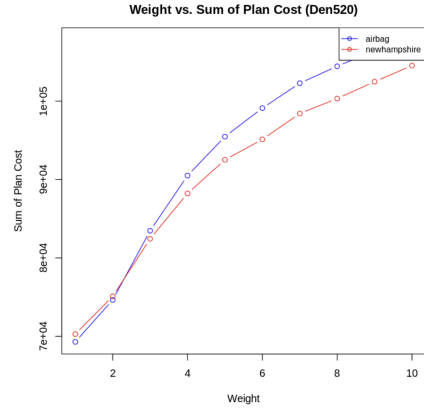


Figure 5.23: Variant comparison for weight vs sum of plan cost in den520d

In hansen, we again observe that New Hampshire has fewer states generated than Airbag. It seems that in both variants, rate of change for number of states generated is stagnant between weights 2 and 3, but increases as the weight approaches 10. For plan cost, we notice that the solution quality for Airbag is lower throughout all the weights. We also notice that the gap in the solution qualities seems to be decreasing as weight is increasing.

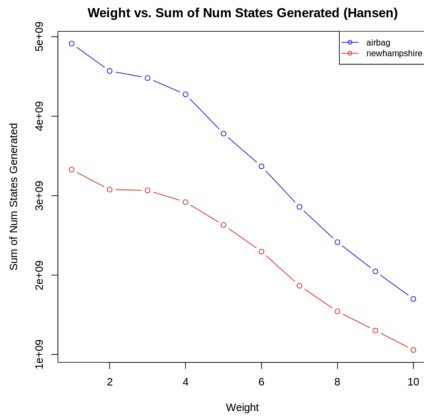


Figure 5.24: Variant comparison of weight vs sum of number of states generated in hansen

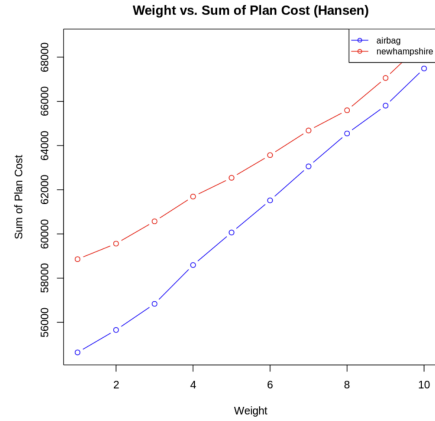


Figure 5.25: Variant comparison of weight vs sum of plan cost in hansen

5.3. WA* ON THE RACETRACK DOMAIN

For the maze, we notice a larger gap between the two variants than in hansen, and less change in number of states generated as weight increases. Airbag seems to decrease the number of states generated more with higher weights than New Hampshire. For plan cost, we notice very little difference between the sum plan cost for the two variants. The New Hampshire plan costs are again the highest, like in hansen. Here, the gap between Airbag plan costs and New Hampshire plan costs and the gap is much larger.

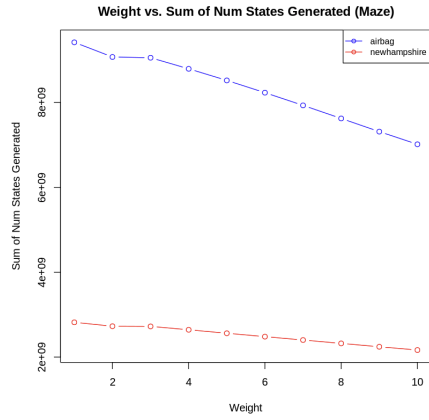


Figure 5.26: Variant comparison of weight vs sum of number of states generated in maze

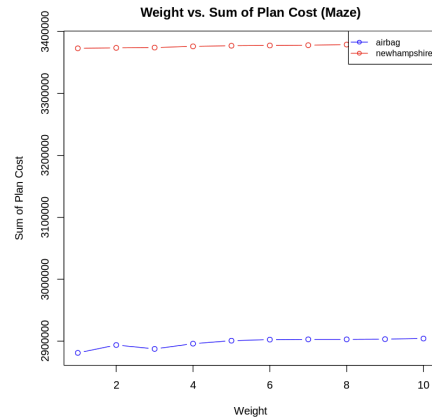


Figure 5.27: Variant comparison of weight vs sum of plan cost in maze

Chapter 6

Conclusion

In this thesis, we aimed to provide a comprehensive view of the racetrack problem in the field of heuristic search. Unlike other classic problems, racetrack is not as well documented, and although it is a benchmark, it has been defined informally. This paper formally defines the racetrack problem and its two variants: New Hampshire and Airbag. It then provides some insight into how the different components of racetrack are implemented. These include the states, actions, goals, and transitions. The Euclidean and Manhattan heuristics, adapted to fit within the racetrack domain, are then explained. Along with those traditional heuristics, we introduced the Chebyshev heuristic. Proofs of consistency and admissibility are provided for each of these heuristic functions. We then outlined alternatives to each of these heuristics that consider acceleration in their estimates [7]. We call these heuristics the Euclidean velocity, Manhattan velocity, and Chebyshev velocity heuristics. We also provided proofs of admissibility and consistency for these velocity heuristics as well. Finally, we analyzed the performance of A^* and WA^* with these heuristics on three different maps within the racetrack domain.

We found that for A^* , the Chebyshev and Chebyshev velocity heuristics are the most informed and generally perform better than the other heuristics. We also found that the New Hampshire variant provides longer solutions than Airbag, because of its extra restriction on collisions. Conversely, the Airbag variant generates more states because it does not avoid collisions. We saw these two properties exaggerated as we transitioned between maps with few collisions (like den520d) to maps with more collisions (like hansen and the maze).

For WA^* , we noticed varying performance between the different heuristics and maps. The Manhattan heuristic offered better performance in these suboptimal search problems, especially in hansen and the maze. The Chebyshev heuristics did not maintain their performance advantage over the other heuristics as weight increased. We did observe that Chebyshev still does the best in some maps with WA^* , but its performance does generally degrade. Chebyshev velocity had a similar result, although it was never the

best heuristic for all tested weights, in any metric. For the two variants of the problem in WA^* , we noticed that New Hampshire mostly maintained a worse solution quality, but fewer states were generated, as in A^* . However, for the den520d map, New Hampshire actually had better solution quality on average than Airbag.

In conclusion, the formal definitions of the domain heuristics provided in this dissertation allow for consistent benchmarking for future research. Our results also show that the Chebyshev and Chebyshev velocity heuristics are well tailored to the racetrack problem, which is an advantage for racetrack benchmarks in general. The differences in the two variants of racetrack can also be studied for future research on dead end states.

While this thesis does provide significant insight into the racetrack domain, several limitations and room for future research are acknowledged.

- In our implementation of racetrack, the agent must land on the finish line to solve a problem. It is also acceptable to allow the agent to cross the finish line as its solution. We left this functionality for future work and research.
- Recall that in Section 3.4.1, we outlined a problem with the collision detection algorithm that we used for this paper. The main problem is that it can be inaccurate in certain cases, causing the agent to not detect certain collisions. We were unable to test racetrack on a random map for this reason. Instead, we used maps where all obstacles were long walls along the map. A more accurate collision detection algorithm could be used for maps with scattered single-unit obstacles. In the future, perhaps interesting insights could be gained by using a better collision detection algorithm on these maps.
- The current algorithm used for Chebyshev velocity requires a heuristic calculation for every single position in the finish line. This will add a lot of overhead for very large finish lines. We experienced this performance problem during our experiments with the maze map. Since a few of the finish lines for this map were 511 units long, each node expansion required a massive computation overhead. In the future, a more efficient implementation of this heuristic would improve the Chebyshev velocity heuristic’s usability.

Bibliography

- [1] Andrew G. Barto, Steven J. Bradtke, and Satinder P. Singh. Learning to act using real-time dynamic programming. *Artif. Intell.*, 72(1-2):81–138, 1995. → pages 1
- [2] Maximilian Fickert, Tianyi Gu, and Wheeler Ruml. New results in bounded-suboptimal search. In *Thirty-Sixth AAAI Conference on Artificial Intelligence, AAAI 2022, Thirty-Fourth Conference on Innovative Applications of Artificial Intelligence, IAAI 2022, The Twelveth Symposium on Educational Advances in Artificial Intelligence, EAAI 2022 Virtual Event, February 22 - March 1, 2022*, pages 10166–10173. AAAI Press, 2022. → pages 28
- [3] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. → pages 7, 8
- [4] Ira Pohl. Heuristic search viewed as path finding in a graph. *Artificial intelligence*, 1(3-4):193–204, 1970. → pages 8
- [5] Roni Stern, Nathan R. Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne T. Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, T. K. Satish Kumar, Eli Boyarski, and Roman Bartak. Multi-agent pathfinding: Definitions, variants, and benchmarks. *Symposium on Combinatorial Search (SoCS)*, pages 151–158, 2019. → pages 28
- [6] N. Sturtevant. Benchmarks for grid-based pathfinding. *Transactions on Computational Intelligence and AI in Games*, 4(2):144 – 148, 2012. → pages 28
- [7] Nathan Sturtevant. [nathansttt/hog2](https://nathansttt.github.io/hog2/), 4 2025. → pages 20, 45
- [8] Stephen Wissow, Fanhao Yu, and Wheeler Ruml. Tunable suboptimal heuristic search. In Ariel Felner and Jiaoyang Li, editors, *Seventeenth International Symposium on Combinatorial Search*, pages 170–178. AIII Press, 2024. → pages 2, 13